

MediSync

Development plan

Revised to match the Software Engineering 1 paper.
Status as of October 4, 2026.

At a glance

MediSync is a clinic management system for the National University Dasmarinas clinic. The Software Engineering 1 paper defines it as two connected surfaces on one database: a patient app for students and employees, and a staff portal for nurses, doctors, dentists, student volunteers and the system administrator.

This plan replaces the earlier plan, which covered the student side only. It maps every functional requirement (FR-01 to FR-16) and every non-functional requirement (NFR-01 to NFR-09) in the paper to a stage, and it lists what is out of scope.

Stage	Scope	Status
0	Project skeleton, split settings, custom User	Done
1	Layout shell, shared templates, icons, dashboard	Done
2	Student accounts: login, profile, settings, password change, data export	Done
2P	Placeholder pages so every tab opens (temporary)	Done
3	Foundation: roles, patients, consent, activity log, session timeout	Next
4	Clinic data, admin, seed data, slot generator	Not started
5	Booking steps 1 to 3	Not started
6	Questionnaire and triage suggestion	Not started
7	Confirmation, queue entry, notifications, cancellation	Not started
8	Staff portal: queue desk, check-in, walk-ins, visitors	Not started
9	Patient records and consultation	Not started
10	Inventory and prescriptions	Not started
11	Medical documents and messaging	Not started
12	History, dashboards, reports, removal of sample data	Not started

Stage	Scope	Status
13	Hardening and release	Not started
B	WebView app wrapper (starts after Stage 7)	Not started

1. Decisions behind this plan

The paper changes several earlier assumptions. These decisions are the result. D3 is confirmed. The others are recommendations that stand unless the team objects.

ID	Decision	Reason	Status
D1	Build a role-based staff portal as Django views. Django admin stays for the System Administrator (accounts, setup data).	The paper gives each role its own screens (sections 2.6 and 3.5.1). Admin cannot give nurses, doctors, dentists and volunteers separate workflows.	Proposed
D2	Rename StudentProfile to Patient with an optional user link, and add a QueueEntry model.	Outside visitors have no account (FR-04). Walk-ins have no appointment (FR-05). The queue needs a suggested and a confirmed priority (FR-06).	Proposed
D3	The mobile app is a WebView wrapper around the Django web app.	One codebase and one layout. All templates and logic are reused.	Confirmed
D4	Triage suggestion comes from a weighted scoring function behind a swappable interface. Staff confirm the final priority.	The paper mentions a machine learning classifier. A pure function is testable now, and a trained model can replace it without changing views.	Proposed
D5	No REST API, DRF, Celery, Channels or JS framework. Django templates, vanilla JavaScript, polling.	The WebView needs no API. Polling every 10 seconds is enough for the queue.	Unchanged
D6	The ID reader, Outlook email, SMS and the school student system are stubs behind service classes.	None are available in development. The paper lists them as dependencies (section 2.7).	Proposed

2. Scope

2.1 In scope (from the paper, section 2.4)

- Role-based access and security, consent before data is collected, activity log.
- Electronic health records: patient files, medical history, vital signs, consultations, dental records, incident reports.
- Smart appointment and triage: schedules, booking, symptom questions, suggested priority.
- Digital queue: queue numbers, priority queue, a separate queue for busy days, walk-ins, ID tap check-in.
- Medicine inventory: stock, receiving, dispensing, low-stock alerts, daily and monthly summaries.
- Prescriptions that deduct stock in the same step.
- Medical certificates, excuse letters, medical passes and go-home forms generated from the consultation record.
- Communication: in-app notifications, Outlook email, SMS, direct messages to a student or guardian.
- Dashboard and monthly reports from stored data.

2.2 Out of scope (do not build)

- Handling or scheduling critical emergencies. The system only organizes cases and never replaces professional judgment (section 3.5.4).
- Diagnosis of any kind.
- Health engagement map and quiz (FR-16). The paper calls it supplementary. It waits until the core stages are done.
- Section 2.8 items: advanced ML forecasting, AI symptom chat, deeper school-system sync, online clearance, automatic reordering, advanced analytics, QR verification, parent portal, offline mode, medical device integration, referral recording.
- Native mobile features, push notifications, and a separate native UI.
- Real patient data. Development and testing use sample data only until the clinic gives approval (section 4.1.8).

2.3 Functional requirements and where they land

ID	Requirement	Stage
FR-01	Users log in to their own account	2, 3
FR-02	View doctor and dentist schedule	4, 5
FR-03	Book an appointment, no double booking	5, 7
FR-04	Staff register an outside visitor	8
FR-05	Confirm arrival by ID tap, or add to the walk-in queue	8
FR-06	Suggested priority, staff review and adjust	6, 8
FR-07	Patients see their queue number	7, 8
FR-08	Store and display the medical record	9
FR-09	Track medicine and receive low-stock alerts	10
FR-10	Review the generated medical certificate	11
FR-11	Dashboard and analytics	12
FR-12	Different access levels per role	3
FR-13	Agree to terms and consent before data is collected	3, 8
FR-14	In-app and Outlook notifications	7, 11
FR-15	Staff send a direct message to a student or parent	11
FR-16	Health map and quiz	Deferred

2.4 Non-functional requirements and where they land

ID	Requirement	How it is met	Stage
NFR-01	Responds quickly in normal use	Indexed queries, select_related, load test on sample data	13
NFR-02	Patient data limited by role	Role checks on every view, object-level scoping	3, 9
NFR-03	Easy for different skill levels	Plain labels, short forms, prototype layout	All
NFR-04	Reliable during clinic hours	Atomic transactions, backups, error pages	13
NFR-05	Logout after inactivity	Idle session expiry, configurable	3
NFR-06	Works on the campus network	LAN deployment guide, no external dependency for core flows	13
NFR-07	Stock and queue always current	Stock changes in one transaction, queue polling	8, 10
NFR-08	Clear error messages	Form validation messages, toasts	All
NFR-09	Data privacy compliance	Consent gate, role access, aggregate-only analytics	3, 12

3. Architecture

3.1 Apps and what each one owns

Rule: **clinic** describes what exists. **appointments** describes what a patient books and where they stand in line. **records** holds clinical data. **inventory** holds stock. **portal** holds the staff screens.

App	Owns	Status
core	Layout, dashboard, help and FAQ, error pages, template tags, icons, toast helper	Exists
accounts	User with role, Patient, UserSettings, ActivityLog, VolunteerAgreement, login, consent, session timeout	Exists, grows in Stage 3
clinic	Practitioner, PractitionerAvailability, VisitReason, Symptom, ClosedDate, slot generator	Stage 4
appointments	Booking flow, Appointment, TriageAssessment, QueueEntry, triage service, queue state	Stages 5 to 8
notifications	Notification, Message, delivery service for in-app, email and SMS	Stages 7 and 11
records	MedicalHistory, Consultation, DentalTreatment, Prescription, PrescriptionItem, MedicalDocument, IncidentReport	Stages 9 to 11
inventory	InventoryItem, InventoryTransaction, stock service, low-stock alerts	Stage 10
portal	Staff screens only (queue desk, patient file, inventory, reports). No models of its own.	Stage 8 onward
engagement	Health map and quiz	Deferred

3.2 Roles and access

Role	Can	Cannot
Student, employee	Own profile and settings, book, see own queue number, own history and released documents	See any other person's data, open the staff portal
Outside visitor	No login. Registered by staff at the desk with consent.	Self-service access
Nurse	Open patient files, record vital signs, run the queue, manage stock, send messages, view reports	Issue prescriptions or certificates
Doctor	Full patient history, consultation notes, prescriptions, approve certificates	Manage accounts
Dentist	Dental history, treatments, dental schedule, referral form	Manage accounts, stock
Student volunteer	Public queue numbers and the daily staff schedule	Open any medical record or stock
System administrator	Accounts, roles, activity log, settings, backups, aggregate reports	View individual patient details

3.3 The paper's 21 tables and the Django models

Paper table	Django model	App	Stage
users	User (adds role)	accounts	3
patients	Patient (renamed from StudentProfile, user optional)	accounts	3
medical_histories	MedicalHistory	records	9
volunteer_agreements	VolunteerAgreement	accounts	3
activity_logs	ActivityLog	accounts	3
clinic_schedules	PractitionerAvailability and ClosedDate (slots are generated)	clinic	4
appointments	Appointment	appointments	7
queue_entries	QueueEntry	appointments	7, 8
consultations	Consultation	records	9
dental_treatments	DentalTreatment	records	9
prescriptions	Prescription	records	10
prescription_items	PrescriptionItem	records	10
medical_documents	MedicalDocument	records	11
incident_reports	IncidentReport	records	9
inventory_items	InventoryItem	inventory	10
inventory_transactions	InventoryTransaction	inventory	10
notifications	Notification	notifications	7
messages	Message	notifications	11
health_quizzes, quiz_questions, quiz_results	Quiz models	engagement	Deferred

We also keep models the paper does not list because the prototype needs them: UserSettings, Practitioner, VisitReason, Symptom, TriageAssessment and FAQ. Practitioner gets an optional link to a User so a doctor who logs in is tied to their schedule.

3.4 Queue design

- A **QueueEntry** is created when a booking is confirmed (status booked, with a queue number) or when a walk-in or visitor is registered (status waiting).
- Statuses: booked, waiting, called, in_consultation, completed, cancelled, no_show.
- Appointment status covers only the booking: confirmed, cancelled, completed, no_show. This replaces the earlier plan, where called and checked_in lived on Appointment.
- appointment on QueueEntry is optional, which supports walk-ins.
- queue_type separates the priority queue from the separate queue for busy days.
- Waiting entries are ordered by confirmed priority (high, medium, low), then by arrival time.
- Double booking stays blocked by a conditional unique constraint on practitioner, date and start time where the appointment is not cancelled.

3.5 Priority flow

- The triage function suggests a priority from the patient's answers (Stage 6) and stores it on the QueueEntry as `suggested_priority`.
- A nurse reviews it with the vital signs and sets `confirmed_priority` (Stage 8). The queue uses the confirmed value only.
- Every change is written to the activity log. The final decision on priority always belongs to staff.
- The classifier is chosen by a setting, so a trained model can replace the weighted scoring later.

4. Stage details

Every stage must also pass the checklist in section 6.

Stage 0 to 2P: Completed work

[Done](#)

GOAL

Project skeleton, layout shell, student accounts and placeholder pages.

WHAT TO BUILD

- Split settings, five app packages, custom User, django-environ.
- Layout shell, components (button, badge, form field, modal, toggle), icon tag, toasts, CSS and JS structure.
- Student login, logout, profile, emergency contact, settings, password change, data export, seed_demo, 21 tests.
- Placeholder pages for queue, history, help and the booking steps. These use sample data from apps/core/mock.py and are deleted by later stages.

DONE WHEN

- Runs on a fresh database after migrate and seed_demo.

TESTS

- 21 tests pass.

Stage 3: Foundation: roles, patients, consent, activity log, session timeout

[Next](#)

GOAL

Put role-based access, one shared patient identity, consent and an audit trail in place before any staff feature exists. Every later stage depends on them. Covers FR-01, FR-12, FR-13, NFR-02, NFR-05, NFR-09.

WHAT TO BUILD

- User.role with choices student, employee, nurse, doctor, dentist, volunteer, admin. A RoleRequiredMixin and decorator that every staff view uses.
- Rename StudentProfile to Patient with a migration that keeps all data. Make user optional. Add patient_type (student, employee, visitor), birth_date, sex, address, guardian_name, guardian_phone. Keep the profile accessor on User so existing code works.
- Consent: consent_agreed and consent_date on Patient. A middleware sends any signed-in patient without consent to a terms page. Visitors' consent is recorded by staff in Stage 8.
- ActivityLog (user, action, module, time, object id) and a log_activity helper. Log login, logout, failed login, consent, role changes, and later every patient record view or edit.
- Idle session timeout from SESSION_IDLE_MINUTES in the environment (default 15, to confirm). The session expiry refreshes on each request. The login page explains an inactivity logout.
- VolunteerAgreement model (user, signed time, status).
- Django admin limited to the admin role and superusers.
- seed_demo adds a sample nurse, doctor, dentist, volunteer, administrator and one visitor patient.

DONE WHEN

- A user can open only the pages for their role.
- A patient without consent can reach only the terms page and logout.
- An idle session expires.
- The activity log records the events above.

TESTS

- Role matrix across existing URLs, consent gate, session timeout, log entries created, rename migration keeps data, a visitor Patient with no user.
- The existing 21 tests still pass.

NOTES

- Minors are identified from `birth_date`. The age threshold and the guardian rule need clinic confirmation (open items).

Stage 4: Clinic data, admin, seed data, slot generator

Not started

GOAL

Move everything the clinic offers into the database and generate bookable slots. Groundwork for FR-02 and FR-03.

WHAT TO BUILD

- `clinic` models: Practitioner (name, role, type doctor or dentist, room, active, optional User link), PractitionerAvailability (weekday, start, end, slot minutes), VisitReason (type, triage weight), Symptom (weight, red-flag field), ClosedDate.
- Admin registration for the System Administrator. The staff portal screen for schedules comes in Stage 8.
- Slot generator as a pure function in `apps/clinic/services/slots.py`. Prototype default: 9:00 to 11:30 and 13:00 to 16:00, every 30 minutes. It skips closed dates, past times and taken times that are passed in.
- `seed_demo` adds the prototype's practitioners, visit reasons and symptoms. Seed availability follows the clinic's real hours: doctor Monday to Friday 9:00 to 18:00, dentist Tuesday, Wednesday and Friday 9:00 to 18:00.

DONE WHEN

- Staff can edit all clinic data in admin.
- Slots for a normal weekday match the expected grid.

TESTS

- Default grid, closed date returns nothing, taken slots removed, past slots removed, no availability returns nothing, weekday rules.

NOTES

- The clinic's lunch break and slot length need confirmation (open items).

Stage 5: Booking steps 1 to 3

Not started

GOAL

Let a student or employee choose a visit type and reason, a practitioner, and a date and time. Nothing is saved yet. Covers FR-02 and part of FR-03.

WHAT TO BUILD

- Booking state lives in the session: type, reason, practitioner, date and slot, answers.
- Views for `/book/`, `/book/practitioner/`, `/book/schedule/` replace the placeholders.
- The schedule shows doctor and dentist availability before booking, which fixes the Facebook-only schedule problem.
- Step guard sends the user back to the first incomplete step.
- Server-side checks: practitioner offers the type, date is open and not past, slot is in the generated list.
- Only account holders book. Visitors are registered by staff.
- Components: stepper, choice card, card. Discard modal clears the session only.

DONE WHEN

- A user can reach the questionnaire through all three steps.
- Edited form values cannot select an invalid practitioner, date or slot.

TESTS

- Step guard, invalid slot, closed date, past date, wrong practitioner type, discard clears session only.

NOTES

- The prototype bug `cancelprio` must not return: discarding a draft never touches a saved appointment.

Stage 6: Questionnaire and triage suggestion

Not started

GOAL

Collect symptoms and severity, then suggest a priority. Covers the suggestion half of FR-06.

WHAT TO BUILD

- `/book/symptoms/`: severity slider, symptom checkboxes including red flags, description with character count, consent checkbox.
- `apps/appointments/services/triage.py` with a `TriageClassifier` interface. `WeightedScoreClassifier` is the default. A `TRIAGE_CLASSIFIER` setting selects the class.
- Inputs: reason weight, severity, symptom weights. Output: score and priority (low, medium, high). A red flag forces high.
- `TriageAssessment` stores the answers and the suggestion.
- `/book/result/` shows the suggested priority and estimated wait. The line 'Triage is queue-priority support only, not a diagnosis' stays visible.
- Red-flag symptoms also show the paper's instruction that urgent cases need in-person help at the clinic.
- Small JavaScript for live severity feedback and the counter.

DONE WHEN

- The server computes priority. The browser never decides it.
- The result page matches the prototype's wording.

TESTS

- Table-driven scoring, red-flag override, threshold boundaries, empty symptom list, maximum severity, missing consent rejected, classifier setting swaps the class.

NOTES

- Keep thresholds as named constants so clinic staff can review them.
- The paper lists untrue answers as a risk. Stage 8 lets a nurse override the suggestion.

Stage 7: Confirmation, queue entry, notifications, cancellation

Not started

GOAL

Turn the session booking into a real appointment safely, create the queue entry, and tell the patient. Covers FR-03, FR-07 (number), FR-14.

WHAT TO BUILD

- Confirm view (POST) creates `Appointment`, `TriageAssessment` and `QueueEntry` inside `transaction.atomic()`. Reference and queue number are assigned in the transaction.
- A taken slot raises the database error. The view shows 'that slot was just taken' with a link back to the schedule.
- `/appointments/<ref>/` confirmation page filtered by the signed-in user. Another user's reference returns 404.
- Notification service with three channels: in-app, email (console now, Outlook later), SMS (stub). It respects `UserSettings` and sends only appointment time and practitioner, never symptoms (section 3.5.9).
- Add to calendar (.ics download). Resend SMS uses the same service.
- Cancel (POST) is free up to 15 minutes before the slot. It cancels the appointment and the queue entry and frees the slot. Reschedule is cancel plus a new booking with the practitioner preselected.

DONE WHEN

- Confirming creates exactly one appointment and one queue entry, or nothing.
- The session booking clears after success.

TESTS

- Double booking blocked, cancelled slot rebookable, atomic rollback, another user's reference returns 404, anonymous redirects to login, notification content excludes symptoms, cancel rules.

NOTES

- Views call the notification service. They never talk to a provider directly.

Stage 8: Staff portal: queue desk, check-in, walk-ins, visitors

Not started

GOAL

Nurses run the clinic day from one screen. Covers FR-04, FR-05, FR-06, FR-07, FR-13 and NFR-07.

WHAT TO BUILD

- Portal shell at `/portal/` with a role-aware sidebar, reusing the existing components.
- Queue board with the priority queue and the busy-day queue, ordered by confirmed priority then arrival, polling every 10 seconds.
- Check-in: a focused input for the ID tap (most USB card readers type the ID like a keyboard) and a search by name or student number as the manual fallback. A booked entry moves from booked to waiting. No appointment creates a walk-in entry.
- Visitor registration: name, contact, birth date, guardian when a minor, and a consent checkbox. Creates a Patient with no user.
- Priority review: shows the suggestion and the patient's answers. The nurse sets the confirmed priority. The change is logged.
- Actions: call next, start consultation, no-show, cancel. Each is logged and the patient is notified when called.
- Schedule management screen for availability and closed dates.
- Volunteer view: public queue numbers and the daily staff schedule only.
- Student side: real `/queue/` page and `/queue/status/` JSON with estimated wait. The prototype's fake 8-second auto-call is gone.

DONE WHEN

- A nurse can take a patient from arrival to called without admin.
- A student sees the position change without reloading.

TESTS

- Ordering by priority then arrival, tap matches appointment, walk-in creates entry, visitor consent recorded, volunteer cannot open records, student cannot open `/portal/`, JSON endpoint returns only the caller's entry.

NOTES

- Replace the placeholder queue view.
- A reader failure must never block check-in. The manual search always works.

Stage 9: Patient records and consultation

Not started

GOAL

Replace paper folders with one record per person. Covers FR-08, NFR-02, NFR-08, NFR-09.

WHAT TO BUILD

- records app: MedicalHistory (personal and family conditions), Consultation (linked to the queue entry and patient, with blood pressure, temperature, pulse, weight, height, oxygen level, purpose, diagnosis, intervention, attending staff), DentalTreatment (tooth number, condition, service, referral needed), IncidentReport.
- Patient search by name or student number with a duplicate check. The unique student ID prevents two records for one student.
- Nurse records vital signs. Doctor sees full history and writes notes. Dentist works the dental record and referral.
- Daily clinic log sheet built from queue time in and time out.
- Forms mirror the clinic's numbered forms (visit log sheet, incident report, health examination, medical records, dentist referral and chart).
- Every view and edit of a patient file is written to the activity log.
- The patient sees selected information read-only: history, prescriptions, certificates, appointments.

DONE WHEN

- One person has one record.
- Each role sees only its own fields.

TESTS

- Role access per field group, duplicate check, activity log on view and edit, student sees only own data, validation messages.

NOTES

- Needs the blank forms from the clinic. Use sample data only.

Stage 10: Inventory and prescriptions

Not started

GOAL

Track medicine automatically and tie it to consultations. Covers FR-09 and NFR-07.

WHAT TO BUILD

- inventory app: InventoryItem (generic name, brand name, category, unit, quantity on hand, minimum stock, expiry) and InventoryTransaction (received, dispensed, adjusted, with remarks).
- One service function changes stock, in a transaction with row locks. Nothing else writes quantities.
- Low-stock alert when quantity reaches or falls below the minimum. It notifies nurses and flags the dashboard.
- Prescription and PrescriptionItem in records. Issuing a prescription deducts stock, records the transaction, updates the patient history and gives a printable prescription.
- Daily summary per medicine (start, dispensed, restock, end) from the transactions.
- Restock screen for nurses. Clear error when stock is too low.

DONE WHEN

- Stock never goes negative.
- A failed prescription leaves stock unchanged.

TESTS

- Atomic deduction, concurrent dispensing, low-stock fires at the threshold, rollback on failure, daily summary equals transactions.

NOTES

- Medicine records use both generic and brand name, as the paper states.

Stage 11: Medical documents and messaging

Not started

GOAL

Issue documents from the consultation and let staff message patients. Covers FR-10, FR-15, FR-14.

WHAT TO BUILD

- MedicalDocument types: medical certificate, excuse letter, medical pass, go-home form, pre-employment certificate. Fields: rest days, remarks, status (draft or released), release time.
- A draft is built only from the consultation record. Staff review and release it. The doctor decides what it says.
- Print-ready page first (browser print to PDF). A PDF library only if the clinic asks.
- The student sees released documents only.
- Message model: staff to a student, or to the guardian when the patient is a minor. Tied to a patient or case, kept on record, sent through the notification service.
- Real notifications dropdown with unread count and mark as read.

DONE WHEN

- A certificate contains only consultation-supported fields.
- Students never see drafts.

TESTS

- Content comes from the consultation, draft hidden from the student, guardian message recorded for a minor, access scoped by role.

NOTES

- Message and SMS content needs clinic approval (section 3.5.9).

Stage 12: History, dashboards, reports, removal of sample data

Not started

GOAL

Replace every remaining sample value. Covers FR-11 and NFR-09.

WHAT TO BUILD

- Student history and dashboard from real appointments and consultations.
- FAQ model with admin or portal editing.
- Staff dashboard: visit volume, busy hours, demographics, common illnesses, certificates released, medicine use, inventory status.
- Monthly summary report from stored data, as a print view and CSV.
- The System Administrator and management see aggregate numbers only.
- Delete `apps/core/mock.py` and all placeholder views. Data export includes appointments and documents.

DONE WHEN

- No page reads sample data.
- Nothing imports `mock.py`.

TESTS

- Aggregates match fixtures, administrator cannot open patient detail, report totals, history filters, export contents.

NOTES

- The prototype's 404 sidebar item stays removed.

Stage 13: Hardening and release

Not started

GOAL

Make the system safe and ready for the clinic network. Covers NFR-01, NFR-04, NFR-06.

WHAT TO BUILD

- Error pages for 404, 403 and 500 in the prototype's style.
- Security sweep: login and role on every view, object-level scoping, CSRF on every POST, no unsafe HTML.
- Permission matrix test across every URL and every role.
- Load check on queue and search with sample data that matches about 100 visits a day.
- Production settings: hosts and secret key from the environment, secure cookies, HSTS, WhiteNoise, Gunicorn, PostgreSQL through `DATABASE_URL`.
- Backup procedure for the System Administrator. LAN deployment guide. README and short user guides with pictures for training.

DONE WHEN

- `manage.py check --deploy` is clean.
- A new teammate can set up from the README alone.

TESTS

- Permission matrix, error pages, backup and restore dry run.

NOTES

- Do not run `seed_demo` on a public server.

GOAL

Give students and employees an app that shows the Django site. Starts after Stage 7 so the student flow is ready.

WHAT TO BUILD

- Small Flutter project using `webview_flutter` that loads the site.
- Custom User-Agent suffix `MediSyncApp`. Django reads it and shows a bottom navigation instead of the sidebar.
- Android back button goes back one page. Downloads such as the .ics file open in the phone's browser.
- A 'cannot reach the clinic server' screen with a retry button. The paper has no offline mode.
- Safe-area padding for the notch and gesture bar.
- Server needs HTTPS or a campus network address. Demos use a tunnel.

DONE WHEN

- The booking flow works inside the app on a real phone.

TESTS

- Manual device checklist: login, booking, queue page, sign out, back button, download link, no-connection screen.

NOTES

- No native features and no push notifications.

5. Suggested timeline

The windows follow the paper's Gantt chart (table 4.6). The paper says dates move to match the adviser's final calendar.

Window	Work	Paper phase
Oct 4 to Oct 14	Stages 3 to 6. Defense demo: login, profile, booking through the triage result. Chapter 6 diagrams and screen layouts.	Begin design, defense preparation
Oct 15 to Nov 30	Stages 7 to 11, plus Track B after Stage 7	Further development and coding (weeks 7 to 12)
Dec 1 to Dec 14	Stage 12 and integration testing	Integration and testing
Dec 15 to Dec 31	Stage 13, deployment preparation, training material	Deploying software
Jan 1 to Jan 21	Clinic feedback, bug fixes	Assessing and bug fixes

The paper names these as the November priorities: role-based access, health records, appointment and triage, digital queue and medicine inventory. The stage order covers them in that sequence.

6. Team rules and definition of done

- Keep the prototype's content, wording, layout and styling. A button that only showed a toast becomes a real feature.
- Validate on the server and back it with database constraints.
- Keep JavaScript to interface behavior. Business rules stay on the server.
- Every staff view checks the role. Every patient query is scoped to what that role may see.
- Commit migration files with the model change that created them.
- Anything secret comes from the environment. Never commit `.env`, the database or credentials.
- Use sample data only. No real patient records until the clinic approves.

A stage is done when:

- `python manage.py check` reports no issues.
- `python manage.py makemigrations --check` reports no missing migrations.
- `python manage.py test apps` passes, including new tests for the stage.
- The new pages work at desktop and phone width and match the prototype.
- The requirements listed for the stage work end to end.

7. Open items that need an answer

Item	Needed by	Ask
Blank copies of the clinic's numbered forms	Stages 9 and 11	Clinic (Nurse Kyle agreed to share them)
Idle logout length (default 15 minutes)	Stage 3	Team and clinic
Age that counts as a minor, and who counts as guardian	Stages 3 and 11	Clinic
Lunch break and slot length for the doctor and dentist	Stage 4	Clinic
Appointment statuses the clinic wants besides confirmed, cancelled, completed	Stage 7	Clinic
Approved notification and SMS message contents	Stages 7 and 11	Clinic
Rules for the separate busy-day queue (when it applies, how it is numbered)	Stage 8	Clinic
Access to the school student information system	Deferred	University IT
Who maintains the system after the project	Stage 13	Clinic and university

8. Old stage numbers and new stage numbers

Earlier documents used a student-side-only plan. Use this table to translate.

Earlier plan	This plan
Stage 3: clinic models, admin, seed data	Stage 4 (Stage 3 is now the foundation)
Stage 4: booking steps 1 to 3	Stage 5
Stage 5: questionnaire and triage	Stage 6
Stage 6: confirmation, appointments, notifications	Stage 7
Stage 7: queue, polling, cancellation, staff call-next	Cancellation in Stage 7. Queue, polling and call-next in Stage 8 (staff portal instead of admin actions).
Stage 8: history, dashboard, notifications, help	Stage 12 (notifications dropdown in Stage 11)
Stage 9: error pages, security, tests, production	Stage 13
Not planned	Stages 8 to 11 (staff portal, records, inventory, documents, messaging) and Track B